

A3-PS027-TC039

IPv6WebsiteandDependencyReadiness Auditor

Akshay Kumar,Anil Kumar,Er. Hari Gobind Phatak
akshaykumar90537@gmail.com

AIORI Hackathon 3

October 10, 2026

Problem Description

An AAAA record on the main domain does not make a website IPv6-ready.

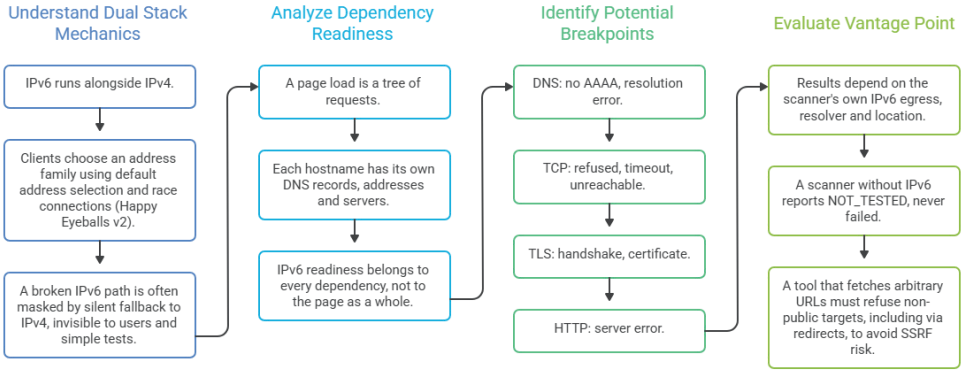
• A website is not fully IPv6-ready simply because its apex domain resolves to an AAAA record.Modern pages pull scripts, stylesheets, fonts, images and API calls from many other hosts (CDNs, analytics, third parties). Any one of them may be IPv4-only, or advertise IPv6 that does not work.
• A DNS-only check cannot see those dependencies, and cannot say whether a failure is in DNS, TCP, TLS or HTTP.
• “AAAA exists” means IPv6 is advertised, not that it works: readiness has to be measured from what a real browser actually loads [1].
• PS-027 asks for: browser-based auditing, a dependency graph, DNS/TCP/TLS/HTTP failure isolation, IPv4 and IPv6 tests of 30 selected sites plus a controlled site seeded with IPv4-only assets, recorded measurement context, and CSV/JSON reports.

Abstract

Webuilta browser-based IPv6 readiness auditor. It loads a page in headless Chromium, records every resource requested, tests each host separately over IPv4 and IPv6 (DNS → TCP → TLS → HTTP), and reports the exact stage at which IPv6 fails, with a dependency graph, measurement context and JSON/CSV exports. 258 automated tests cover each failure stage; real IPv6 measurements are taken on a dual-stack VPS.

Background

Why readiness must be measured per dependency and per address family



Literature & RFC Review

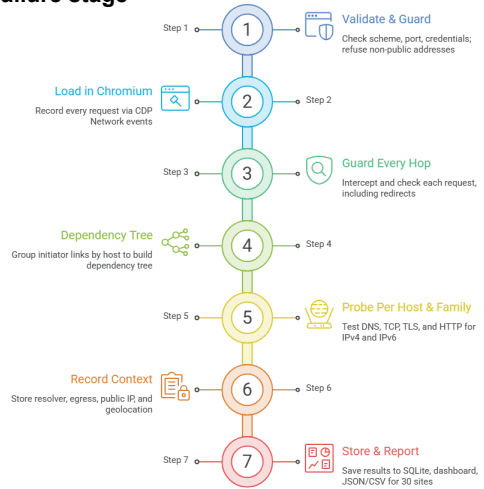
Only sources that shaped a design decision are listed.

Source	What it contributes	How it is used here
RFC 8200 [2]	IPv6 specification; AAAA / IPv6 addressing.	The protocol under test.
RFC 6724 [5], RFC 8305 [6]	Default address selection; Happy Eyeballs v2.	Fallback hides IPv6 failures, so each family is probed separately instead of observing the browser's choice.
RFC 3986 [7], WHATWG URL [8]	URI syntax; browser host parsing (accepts legacy numeric IPv4 forms such as 2130706433).	URL validation; explicit normalisation before the SSRF check.
RFC 6890 [9], RFC 6052 [10]	Special-purpose address ranges; IPv4-embedded IPv6 (NAT64).	Non-public blocklist, including embedded-IPv4 forms.
CWE-918 [11]	Threat model for tools that fetch user-supplied URLs.	Redirect, rebinding and limit controls.
Chrome DevTools Protocol [12]; Playwright #2617, #34994 [13]	Network events (what loaded) and Fetch interception; Playwright does not route redirect hops.	Resource capture via Network domain; per-hop SSRF guard via Fetch domain.
webres6 [1]	Browser-based IPv6 reachability auditing of pages and dependencies.	Concept and requirements; implementation and UI are independent.

Proposed Solution

Real browser load + per-family probing + an explicit failure stage

1. **Validate & guard.** Scheme, port, credentials; resolve and refuse non-public addresses (incl. legacy numeric IPv4 forms).
2. **Load in Chromium (Playwright).** Every request is recorded via CDP Network events: URL, type, initiator, protocol, remote IP, status.
3. **Guard every hop.** CDP Fetch interception checks each request including redirect targets; redirect-depth and host caps apply.
4. **Dependency tree** from initiator links, grouped by host.
5. **Probe per host:port and per family** (IPv4, IPv6): DNS → TCP → TLS (verified) → HTTP. First failing stage is the result; later stages are NOT_REACHED.
6. **Record context** (resolver, egress, public IP, geolocation; "Unknown" if unavailable). No scanner IPv6 → NOT_TESTED, not a failure.
7. **Store & report:** SQLite, dashboard, JSON/CSV; 30-site batch + controlled site.



Optimization Proposed

Baseline (DNS-only check) compared with this solution

Aspect	Before: DNS-only check	After: this solution
Evidence of readiness	An AAAA record exists.	Actual browser load plus per-family DNS → TCP → TLS → HTTP tests.
Dependencies seen	The main domain only.	Every request Chromium made (JS, CSS, fonts, images, XHR/fetch, iframes) as a dependency tree.
Failure output	IPv6 = true / false.	Failure stage and subtype (e.g. TCP_FAILURE / REFUSED). NOT_TESTED and NOT_REACHED are kept distinct from real failures.
Probing cost	One probe per resource.	One per host:port:family (e.g. 200 resources on 15 hosts: 30 probes, not 400).
Redirect safety	Playwright route() guard never sees redirect hops.	CDP Fetch interception on every hop, a 15-hop cap, and a post-connection remote-IP check.
Resource limits	Unbounded hosts and request size.	Caps: 100 hosts/scan, 1 MiB request body, per-stage timeouts, bounded concurrency.
Reports	Raw pass/fail.	JSON/CSV with IPv4 and IPv6 side by side; secrets redacted; CSV formula-injection safe.

Solution Architecture & Design

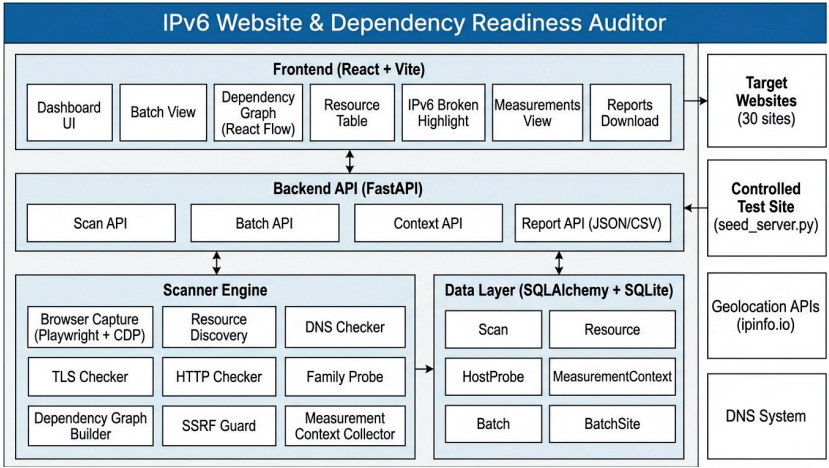


Figure:System architecture (UML component diagram)

Data Flow Diagram

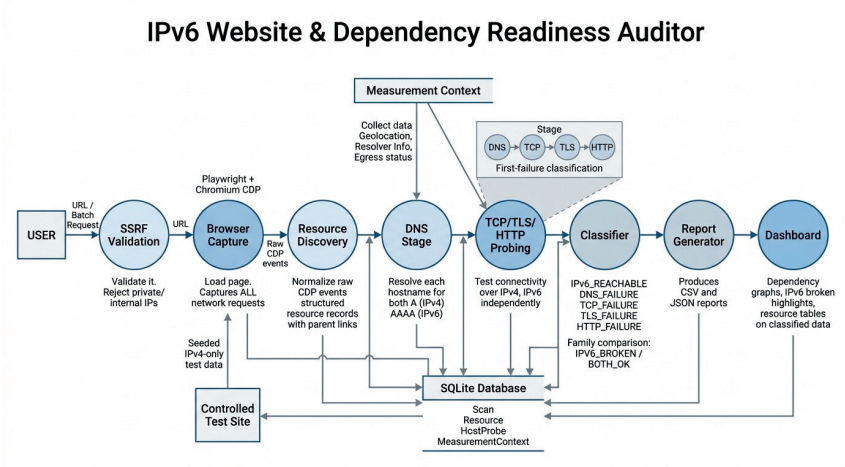


Figure:Data Flow Diagram (DFD)

Deployment Timeline

Incremental phases, each checked against PS-027 and covered by automated tests

#	Milestone	Status
1	Browser capture, DNS, TCP/TLS/HTTP failure classification, SQLite, SSRF guard	Done
2	Dependency graph (API + React Flow); controlled IPv4-only site with seeded-failure detection tests	Done
3	30-site batch runner (sites.json, concurrency, per-site timeout); JSON/CSV reports	Done
4	Dashboard (batch UI, “works over IPv4, fails over IPv6” view); security audit and hardening	Planned
5	Deploy on a dual-stack VPS; publish the controlled site with real DNS (A-only and AAAA-but-broken hosts) and certificates	Planned
6	Run the 30-site and controlled-site measurements from the VPS; export CSV/JSON	Planned
7	Final technical report and live demo with pseudocode walkthrough	Planned

Status of testing: 217 backend + 6 controlled-site + 35 frontend automated tests pass. In the local tests the IPv6 path of each seeded failure is simulated with a second IPv4 address; authoritative IPv6 results come from the VPS run (milestones 5–6). Container configuration for the controlled site is validated structurally, not yet run.

Bibliography

[1] SAP, “webres6,” https://github.com/SAP/webres6.
[2] S. Deering, R. Hinden, “RFC 8200: Internet Protocol, Version 6 (IPv6) Specification,” IETF, July 2017.
[3] D. Thaler (Ed.), R. Draves, A. Matsumoto, T. Chown, “RFC 6724: Default Address Selection for IPv6,” IETF, Sept. 2012.
[6] D. Schinazi, T. Pauly, “RFC 8305: Happy Eyeballs Version 2: Better Connectivity Using Concurrency,” IETF, Dec. 2017.
[7] T. Berners-Lee, R. Fielding, L. Masinter, “RFC 3986: Uniform Resource Identifier (URI): Generic Syntax,” IETF, Jan. 2005.
[8] WHATWG, “URL Standard,” https://url.spec.whatwg.org/.
[9] M. Cotton, L. Vegoda, R. Bonica (Ed.), B. Haberman, “RFC 6890: Special-Purpose IP Address Registries,” IETF, Apr. 2013.
[10] C. Bao, C. Huitema, M. Bagnulo, M. Boucadair, X. Li, “RFC 6052: IPv6 Addressing of IPv4/IPv6 Translators,” IETF, Oct. 2010.
[11] MITRE, “CWE-918: Server-Side Request Forgery (SSRF),” https://cwe.mitre.org/data/definitions/918.html.
[12] Google, “Chrome DevTools Protocol: Network and Fetch domains,” https://chromedevtools.github.io/devtools-protocol/.
[13] Microsoft, Playwright pull request #2617 and issue #34994 (route handlers and redirects), https://github.com/microsoft/playwright.

Thank You for Your Attention!